Week 7 Asynchronous Materials Guide - Advanced List Operations

This Week's Focus

Note: No in-person lecture this week. This is a focused review and extension of Week 6 concepts.

- Reinforcing: Methods that modify vs methods that return values
- Understanding list references and creating proper copies
- Converting between strings and lists with split() and join()
- Practical patterns combining list operations

Additional Practice Resources

LearnPython.org - Lists

https://www.learnpython.org/en/Lists

- Interactive tutorial with exercises
- Practice list operations in browser
- Complete the exercise at the end

W3Schools - List Methods Reference

https://www.w3schools.com/python/python_lists_methods.asp

- Quick reference for all list methods
- Short examples for each method
- Good for looking up syntax

Review: Methods That Modify vs Return

This is one of the most important concepts with lists. Week 6 covered this, but it's critical enough to reinforce with more examples.

Understanding Modify vs Return

Some methods change the list directly and return `None`. Others return a new value without changing the original. Confusing these causes bugs.

Methods That Modify In Place

```
# These change the list and return None
numbers = [3, 1, 4, 1, 5]

result = numbers.append(9)
print(f"append() returns: {result}")  # None
print(f"List is now: {numbers}")  # [3, 1, 4, 1, 5, 9]
print()

result = numbers.sort()
print(f"sort() returns: {result}")  # None
print(f"List is now: {numbers}")  # [1, 1, 3, 4, 5, 9]
print()

result = numbers.reverse()
print(f"reverse() returns: {result}")  # None
print(f"List is now: {numbers}")  # [9, 5, 4, 3, 1, 1]

# Modifying methods: append, insert, extend, remove, pop,
#                    sort, reverse, clear
```

Functions/Methods That Return Values

```
# These return new values WITHOUT changing the list
numbers = [3, 1, 4, 1, 5]

result = len(numbers)
print(f"len() returns: {result}")  # 5
print(f"List unchanged: {numbers}")  # [3, 1, 4, 1, 5]
print()
```

```
result = sorted(numbers)
print(f"sorted() returns: {result}")  # [1, 1, 3, 4, 5]
print(f"List unchanged: {numbers}")  # [3, 1, 4, 1, 5]
print()

result = max(numbers)
print(f"max() returns: {result}")  # 5
print(f"List unchanged: {numbers}")  # [3, 1, 4, 1, 5]

# Returning functions/methods: len, sorted, max, min, sum,
#                              index, count, copy
```

Most Common Mistake

```
# WRONG - sort() returns None!
numbers = [3, 1, 4]
sorted_numbers = numbers.sort()
print(sorted_numbers)  # None!
print(numbers)  # [1, 3, 4] - but you lost the reference

# CORRECT - Use sort() for in-place sorting
numbers = [3, 1, 4]
numbers.sort()
print(numbers)  # [1, 3, 4]

# CORRECT - Use sorted() to get new sorted list
numbers = [3, 1, 4]
sorted_numbers = sorted(numbers)
print(f"Original: {numbers}")  # [3, 1, 4]
print(f"Sorted: {sorted_numbers}")  # [1, 3, 4]
```

List References and Copying

Week 6 showed that `copy = original[:]` creates a copy. This section explores why copying matters and the problems that arise if you don't copy when needed.

W3Schools - Copy Lists

https://www.w3schools.com/python/python_lists_copy.asp

- Simple explanation of list copying
- Three methods: copy(), list(), and slicing
- Interactive examples you can try

Programiz - Python List copy()

https://www.programiz.com/python-programming/methods/list/copy

- Clear examples of copy() method
- Shows difference between copy and reference
- Beginner-friendly explanations

The Reference Problem

```
# Assignment does NOT create a copy
original = [1, 2, 3]
reference = original  # Both variables point to SAME list

print(f"Original: {original}")
print(f"Reference: {reference}")
print()

# Modifying through reference affects original
reference.append(4)
print(f"After reference.append(4):")
print(f"Original: {original}")  # [1, 2, 3, 4] - Changed!
print(f"Reference: {reference}")  # [1, 2, 3, 4]
print()

# Modifying original affects reference
original[0] = 999
print(f"After original[0] = 999:")
print(f"Original: {original}")  # [999, 2, 3, 4]
print(f"Reference: {reference}")  # [999, 2, 3, 4]
print()

# They are literally the same object
print(f"Same object? {original is reference}")  # True
```

Creating Independent Copies

```
# Method 1: Slicing creates a copy
original = [1, 2, 3]
copy1 = original[:]

copy1.append(4)
print(f"Original: {original}")  # [1, 2, 3] - Unchanged
print(f"Copy: {copy1}")  # [1, 2, 3, 4]
print()

# Method 2: copy() method
original = [1, 2, 3]
copy2 = original.copy()

copy2.append(4)
print(f"Original: {original}")  # [1, 2, 3] - Unchanged
print(f"Copy: {copy2}")  # [1, 2, 3, 4]
print()

# Method 3: list() constructor
original = [1, 2, 3]
copy3 = list(original)

copy3.append(4)
print(f"Original: {original}")  # [1, 2, 3] - Unchanged
print(f"Copy: {copy3}")  # [1, 2, 3, 4]

# All three methods create independent copies
# Use whichever you find most readable
```

When Copying Matters

```
# Example 1: Preserving original data
student_scores = [85, 92, 78, 95, 88]
print(f"Original order: {student_scores}")

# Need to sort for analysis but keep original order
sorted_scores = student_scores.copy()
sorted_scores.sort()

print(f"Sorted: {sorted_scores}")
```

```python
print(f"Original preserved: {student_scores}")
print()

# Example 2: Comparing before and after
inventory = [10, 25, 5, 30, 15]
print(f"Inventory at start: {inventory}")

# Save snapshot before processing
snapshot = inventory.copy()

# Simulate sales
inventory[0] -= 5
inventory[2] -= 3
inventory[4] -= 10

print(f"Inventory at end: {inventory}")
print(f"Original snapshot: {snapshot}")

# Calculate changes
for i in range(len(inventory)):
    change = snapshot[i] - inventory[i]
    print(f"Item {i}: sold {change} units")
```

Common Bug: Modifying While Iterating

```python
# WRONG - Modifying list while going through it causes problems
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num % 2 == 0:
        numbers.remove(num)  # BAD! Changes list during iteration
print(numbers)  # May not work as expected

# CORRECT - Work on a copy
numbers = [1, 2, 3, 4, 5]
for num in numbers[:]:  # Iterate over copy
    if num % 2 == 0:
        numbers.remove(num)  # Modifying original is now safe
print(numbers)  # [1, 3, 5]

# BETTER - Build new list
numbers = [1, 2, 3, 4, 5]
```

```
odd_numbers = []
for num in numbers:
    if num % 2 != 0:
        odd_numbers.append(num)
print(odd_numbers)  # [1, 3, 5]
```

Converting Between Strings and Lists

Week 6 briefly showed `split()` for creating lists. This section explores split() and join() in depth for text processing.

W3Schools - Python split()

https://www.w3schools.com/python/ref_string_split.asp

- Simple split() method examples
- Interactive "Try it Yourself" editor
- Different separator examples

W3Schools - Python join()

https://www.w3schools.com/python/ref_string_join.asp

- Simple join() method examples
- How to combine list items into string
- Examples with different separators

freeCodeCamp - split() and join() Explained

https://www.freecodecamp.org/news/python-string-split-and-join-methods-explained-with-examples/

- Detailed tutorial with examples
- Step-by-step explanations
- Practical use cases

Programiz - Python split()

https://www.programiz.com/python-programming/methods/string/split

- Complete split() reference
- Optional parameters explained
- Practice examples

SPLIT - String to List

```
# split() breaks string into list
sentence = "Python is amazing"
words = sentence.split()  # Splits on whitespace by default
print(f"Sentence: {sentence}")
print(f"Words: {words}")
print(f"Word count: {len(words)}")
print()

# Split on custom separator
csv_data = "apple,banana,cherry,date"
fruits = csv_data.split(",")
print(f"CSV: {csv_data}")
print(f"Fruits list: {fruits}")
print()

# Split on different characters
path = "folder/subfolder/file.txt"
parts = path.split("/")
print(f"Path: {path}")
print(f"Parts: {parts}")
print()

# Limit number of splits
text = "one:two:three:four:five"
limited = text.split(":", 2)  # Split only twice
print(f"Limited split: {limited}")  # ['one', 'two', 'three:four:five']
print()

# Split lines of text
multi_line = "line1\nline2\nline3"
lines = multi_line.split("\n")
print(f"Lines: {lines}")

# Common pattern: strip whitespace from each item
messy = "  apple  ,  banana,cherry  "
items = messy.split(",")
```

```
clean_items = []
for item in items:
    clean_items.append(item.strip())
print(f"Cleaned: {clean_items}")
```

JOIN - List to String

```
# join() combines list items into single string
words = ["Python", "is", "awesome"]
sentence = " ".join(words)  # Join with spaces
print(f"Words: {words}")
print(f"Sentence: {sentence}")
print()

# Join with different separators
fruits = ["apple", "banana", "cherry"]

csv = ",".join(fruits)
print(f"CSV: {csv}")

hyphenated = "-".join(fruits)
print(f"Hyphenated: {hyphenated}")

piped = " | ".join(fruits)
print(f"Piped: {piped}")
print()

# Join with no separator
letters = ["H", "e", "l", "l", "o"]
word = "".join(letters)
print(f"Letters: {letters}")
print(f"Word: {word}")
print()

# Join requires all strings
numbers = [1, 2, 3, 4, 5]
# This errors: result = "-".join(numbers)

# Convert to strings first
number_strings = []
for num in numbers:
    number_strings.append(str(num))
result = "-".join(number_strings)
print(f"Number string: {result}")
```

Practical Pattern: Text Processing

```
# Process user input: comma-separated items
user_input = "  apple, banana  , cherry  ,  date"
print(f"Raw input: '{user_input}'")

# Split and clean
items = user_input.split(",")
cleaned = []

for item in items:
    trimmed = item.strip()
    if trimmed:  # Only add non-empty items
        cleaned.append(trimmed)

print(f"Processed: {cleaned}")
print(f"Count: {len(cleaned)} items")

# Display formatted
formatted = ", ".join(cleaned)
print(f"Formatted output: {formatted}")
```

Practical Pattern: CSV Processing

```
# Read CSV data
data = "85,92,78,95,88"
print(f"CSV data: {data}")

# Convert to list of integers
score_strings = data.split(",")
scores = []
for s in score_strings:
    scores.append(int(s))

print(f"Scores: {scores}")
print(f"Average: {sum(scores) / len(scores):.1f}")
print(f"Highest: {max(scores)}")
print(f"Lowest: {min(scores)}")
print()
```

```
# Format results back to CSV
results = [
    str(min(scores)),
    str(max(scores)),
    f"{sum(scores) / len(scores):.1f}"
]
result_csv = ",".join(results)
print(f"Results CSV: {result_csv}")
```

Practical Patterns

Pattern: Building and Formatting Lists

```
# Start with empty list, build it up
menu_items = []
menu_items.append("Pizza")
menu_items.append("Burger")
menu_items.append("Salad")
menu_items.append("Pasta")

print("Menu Items:")
for i in range(len(menu_items)):
    print(f"{i+1}. {menu_items[i]}")
print()

# Sort and display
menu_items.sort()
print("Alphabetical Menu:")
numbered = []
for i in range(len(menu_items)):
    numbered.append(f"{i+1}. {menu_items[i]}")
print("\n".join(numbered))
```

Pattern: Filtering Lists

```
# Keep only items that meet criteria
all_scores = [45, 78, 92, 55, 88, 63, 95, 72]
print(f"All scores: {all_scores}")

# Build new list with passing scores
```

```
passing_scores = []
for score in all_scores:
    if score >= 60:
        passing_scores.append(score)

print(f"Passing scores: {passing_scores}")
print(f"Pass rate: {len(passing_scores)}/{len(all_scores)}")

# Failing scores
failing_scores = []
for score in all_scores:
    if score < 60:
        failing_scores.append(score)

print(f"Failing scores: {failing_scores}")
```

Pattern: List Statistics

```
# Analyze list data
temperatures = [72, 68, 75, 71, 69, 74, 70]
print(f"Temperatures: {temperatures}")

# Calculate statistics
avg = sum(temperatures) / len(temperatures)
highest = max(temperatures)
lowest = min(temperatures)
range_temp = highest - lowest

print(f"Average: {avg:.1f}°F")
print(f"Highest: {highest}°F")
print(f"Lowest: {lowest}°F")
print(f"Range: {range_temp}°F")

# Count above/below average
above = 0
below = 0
for temp in temperatures:
    if temp > avg:
        above += 1
    elif temp < avg:
        below += 1

print(f"Above average: {above} days")
print(f"Below average: {below} days")
```

Pattern: Removing Duplicates

```python
# Remove duplicate values from list
numbers = [1, 2, 3, 2, 4, 1, 5, 3]
print(f"Original: {numbers}")

# Build new list with only unique items
unique = []
for num in numbers:
    if num not in unique:
        unique.append(num)

print(f"Unique (order preserved): {unique}")

# Alternative using set (order not preserved)
numbers = [1, 2, 3, 2, 4, 1, 5, 3]
unique2 = list(set(numbers))
print(f"Unique (set method): {unique2}")
```

Pattern: Finding Items in Lists

```python
# Search for items in list
students = ["Alice", "Bob", "Charlie", "Diana", "Eve"]
print(f"Students: {students}")

# Check if student exists
search_name = "Charlie"
if search_name in students:
    position = students.index(search_name)
    print(f"{search_name} found at position {position}")
else:
    print(f"{search_name} not found")

# Find all students starting with specific letter
letter = "C"
matching = []
for student in students:
    if student.startswith(letter):
        matching.append(student)

print(f"Students starting with '{letter}': {matching}")
```

Quick Reference

Modify vs Return Summary

```
# Methods that MODIFY and return None:
my_list.append(item)
my_list.insert(i, item)
my_list.extend(list2)
my_list.remove(item)
my_list.pop()
my_list.sort()
my_list.reverse()
my_list.clear()

# Functions/methods that RETURN values:
len(my_list)
sorted(my_list)    # Returns new sorted list
max(my_list)
min(my_list)
sum(my_list)
my_list.index(item)
my_list.count(item)
my_list.copy()     # Returns new copy
```

Copying Lists

```
# Three ways to create independent copies:
copy1 = original[:]
copy2 = original.copy()
copy3 = list(original)

# Reference (NOT a copy):
reference = original  # Same object!
```

String/List Conversion

```
# String to list:
words = text.split()           # Split on whitespace
items = text.split(",")        # Split on comma
chars = list(text)             # String to chars
```

```
# List to string:
text = " ".join(words)         # Join with spaces
csv = ",".join(items)          # Join with commas
text = "".join(chars)          # Join with no separator
```